

CrowdArb — Technical Architecture

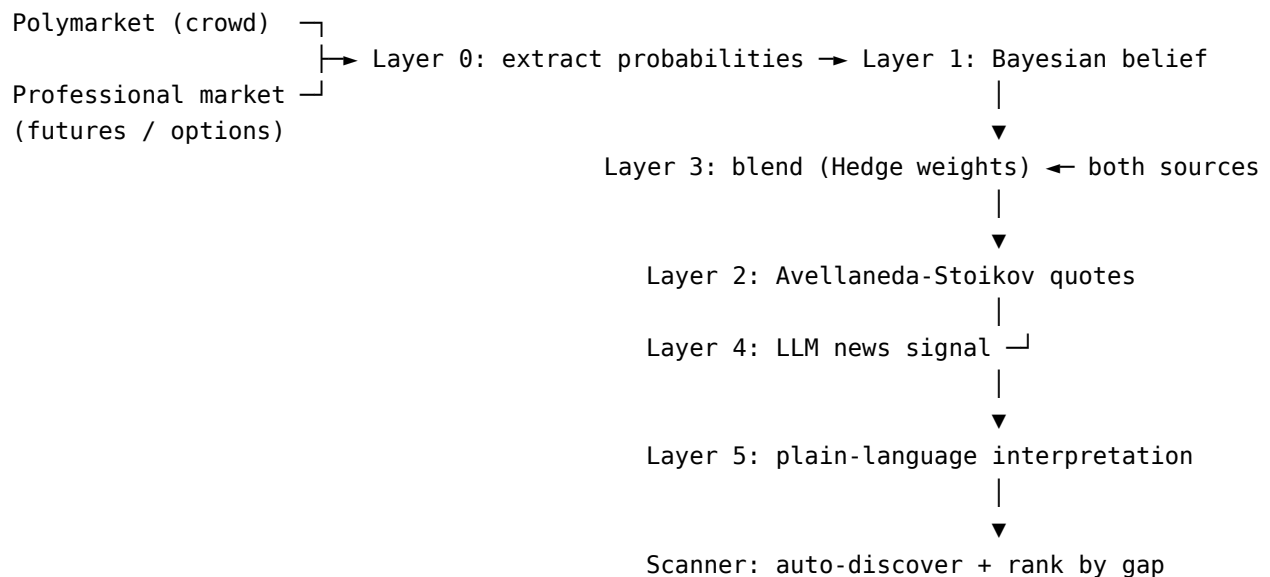
Aris Rault

July 2026

Live system: <https://crowdarb.streamlit.app> **Repository:** <https://github.com/Arisrlt46/crowdarb>

A layered cross-market calibration engine. Each layer has a single responsibility and feeds the next. Adding a new market is a small, isolated change thanks to the MarketPair interface.

1 SYSTEM OVERVIEW



2 LAYER 0 — PROBABILITY EXTRACTION

Pulls the implied probability of the same event from two independent sources.

- **Fed rates:** algebra on CME Fed funds (ZQ) futures. Recovers the post-meeting rate from the month-average rate; converts to $P(\text{cut})$. Uses the next-month contract near month-end for numerical stability.
- **Crypto:** Black-Scholes $N(d_2)$ on the asset's price level, using Deribit implied volatility (90-day realized fallback).
- Shared `layer0_bs.py` holds the Black-Scholes functions so crypto markets reuse one tested implementation.

3 LAYER 1 — BAYESIAN BELIEF UPDATING

A Beta-Bernoulli conjugate model maintains a running probability as simulated order flow arrives. YES fills increment α , NO fills increment β ; the posterior mean $\alpha/(\alpha+\beta)$ is the current belief, and $\alpha+\beta$ is its confidence.

4 LAYER 2 — AVELLANEDA-STOIKOV MARKET-MAKING

Computes an inventory-adjusted reservation price and an optimal two-component spread, then quotes bid/ask. Shifts quotes against inventory to control one-sided risk. Headline result: 37–63% P&L variance reduction vs naive quoting.

5 LAYER 3 — CROSS-MARKET CALIBRATION

Bayesian model averaging with online Hedge weight learning blends the two probability sources, automatically up-weighting whichever has been better calibrated. Headline result: up to 9.6% Brier improvement vs the best single source.

6 LAYER 4 — LLM NEWS SIGNAL

Converts a news headline into a likelihood ratio via the Anthropic API, then applies a Bayesian odds update. Uses a Fed-specific prompt for rate markets and a crypto-specific prompt for digital assets, selected by market type.

7 LAYER 5 — INTERPRETATION ENGINE

Reads the output of Layers 0–4 and produces plain-language explanations of every figure.

Three entry points:

- **interpret(sig) → Interpretation** — pure, deterministic, no API calls. Classifies the crowd-vs-professional gap against three thresholds (3 / 8 / 15 pp → aligned / modest / notable / large), sets direction, and builds one sentence per fact present in the signals bundle: source probabilities, weighted blend, reservation-price tilt vs mid, spread, and any LLM likelihood ratio.
- **compact_read(sig) → str** — one-line scanner label, e.g. "Crowd 21 pts high".
- **narrate(sig, interp, client) → str** — optional LLM paragraph (2–3 sentences, plain English, no jargon) for explicit user requests. One API call per click; cached in session state.

GLOSSARY exports 10 jargon-free definitions wired into `st.metric(help=...)` tooltips and a dashboard expander.

8 SCANNER — REAL-TIME AUTO-DISCOVERY

`discover_markets()` scans ~2,100 live Polymarket markets, classifies each (crypto level / rate decision / equity level / unsupported), parses strike and date, filters by volume, expiry, and a Black-Scholes sanity band, builds a `MarketPair` for each survivor, and ranks them by crowd-vs-professional gap. The dashboard renders the ranked table (with a "Read" column from Layer 5), metric tooltips, and a per-market detail panel with a full Layer 5 interpretation.

9 THE MARKETPAIR INTERFACE

The architectural core. Every market implements three methods:

- `get_polymarket_probability() -> float`
- `get_professional_probability() -> float`
- `metadata() -> dict (name, description, resolution date, type)`

`run_crowdarb(market)` runs Layers 1–5 on any `MarketPair` with no market-specific logic in the pipeline body. Adding a market is one class of roughly 20 lines; the entire engine then works on it automatically.

10 KEY FILES

File	Role
layer0_data.py, layer0_fedwatch.py	Fed: Polymarket + CME extraction
layer0_bs.py	Shared Black-Scholes functions
layer0_bitcoin.py, layer0_ethereum.py	Crypto extraction
layer0_classifier.py	Market classification + strike/date parsing
layer1_belief.py, layer1_backtest.py	Bayesian belief + backtest
layer2_avellaneda.py	Market-making
layer3_calibration.py	Hedge blending
layer4_llm_signal.py	LLM news signal (per-market prompts)
layer5_interpret.py	Interpretation engine + GLOSSARY
market_pair.py	MarketPair interface + discover_markets()
dashboard_scanner.py	Streamlit scanner (deployed app)

11 REPRODUCIBILITY AND ENGINEERING NOTES

- Backtests use fixed seeds and a fixed initial price.
- PROBLEMS_AND_SOLUTIONS.md logs six bugs and one design decision with root-cause analysis.
- CHANGELOG.md documents the Layer 5 addition.
- Deployed on Streamlit Community Cloud; the Anthropic API key is stored in Streamlit secrets, never committed.
- A monthly API spend cap and disabled auto-reload bound the cost of the public LLM feature. The Layer 5 narrative call is opt-in and cached per market per session.